

Migraciones con Alembic

Guía práctica de cómo se gestiona el esquema de la base de datos en este proyecto. **El esquema lo gestiona exclusivamente Alembic**. Antes de levantar la API por primera vez —o en cada deploy— hay que aplicar las migraciones.

Todos los comandos van con el prefijo `uv run` (usa el `.venv` del proyecto sin activarlo). Si activaste el entorno (`source .venv/bin/activate`), puedes omitir `uv run`.

Cómo está configurado aquí

- **Modo async:** Alembic se inicializó con el template `async`, así reutiliza el mismo driver `asyncmy` que la app.
- **alembic.ini:** config general. La `sqlalchemy.url` **no** se fija aquí; la provee `env.py`.
- **alembic/env.py:** el "pegamento" con la app. En concreto:
 - `target_metadata = Base.metadata` → Alembic conoce tus modelos (`app/models.py`), necesario para el `--autogenerate`.
 - La URL de conexión sale de `app.core.settings` (la misma que la app), y se puede sobrescribir con la variable de entorno `DATABASE_URL`.
- **alembic/versions/:** aquí viven los archivos de migración. Ya hay uno inicial, `create users table`.

Conceptos en 30 segundos

- **Revisión (revision):** un archivo de migración en `alembic/versions/`, con un `revision` (id único) y un `down_revision` (a quién sigue). Encadenadas forman un historial lineal.
- **head:** la última revisión de la cadena. **base:** el estado vacío (antes de cualquier migración).
- **alembic_version:** tabla que Alembic crea en tu DB para recordar en qué revisión está el esquema.
- **upgrade / downgrade:** aplicar o revertir migraciones.

Comandos del día a día

```
# Aplicar TODAS las migraciones pendientes (lo más habitual)
uv run alembic upgrade head

# Ver en qué revisión está la base de datos ahora mismo
uv run alembic current

# Ver el historial de migraciones (encadenado)
uv run alembic history --verbose
```

```
# Ver cuál es la última revisión disponible en el código
uv run alembic heads

# Revertir la última migración
uv run alembic downgrade -1

# Volver al estado vacío (revierte todo)
uv run alembic downgrade base

# Aplicar/revertir hasta una revisión concreta (por su id)
uv run alembic upgrade fdd812fef97f
uv run alembic downgrade fdd812fef97f

# Ver el SQL sin ejecutarlo (modo offline), útil para revisar o para DBAs
uv run alembic upgrade head --sql
```

El flujo con `--autogenerate` (lo importante)

`--autogenerate` compara **tus modelos** (`Base.metadata`, lo que *debería* existir) contra el **estado real de la base de datos** (lo que *existe*) y escribe una migración con las diferencias. Es la forma normal de crear migraciones cuando cambias los modelos.

Receta paso a paso

```
# 1. Asegúrate de que la DB está al día ANTES de autogenerar
uv run alembic upgrade head

# 2. Cambia tus modelos en app/models.py
#    (p. ej. añades una columna `telefono` a User)

# 3. Genera la migración comparando modelos vs DB
uv run alembic revision --autogenerate -m "add telefono a user"

# 4. ABRE el archivo generado en alembic/versions/ y REVÍSALO (ver abajo)

# 5. Aplícala
uv run alembic upgrade head
```

El paso 1 importa: autogenerate compara contra la DB conectada. Si la DB no está en `head`, las diferencias que detecte estarán mezcladas con migraciones que aún no aplicaste y saldrá basura.

⚠ SIEMPRE revisa la migración autogenerada

Autogenerate es una ayuda, **no** es infalible. Detecta bien:

- tablas y columnas nuevas o eliminadas,
- cambios de nulabilidad,
- índices y unique constraints,
- (con `compare_type=True`, ya activado aquí) cambios de tipo de columna.

Pero **NO detecta ni acierta** en:

- **renombrar** una tabla o columna → lo ve como "borra la vieja + crea la nueva" (perderías datos). Hay que editarlo a mano con `op.alter_column(..., new_column_name=...)`.
- **cambios de `server_default`** y algunos defaults, según el caso.
- **datos**: autogenerate solo toca el esquema. Si necesitas migrar/backfillar datos, se escribe a mano en el `upgrade()`.
- objetos que no están en la metadata (vistas, triggers, tablas de otras bases...). Fíjate: en este proyecto la metadata solo tiene los modelos de la app, así que si autogeneras contra una DB con tablas ajenas, Alembic querrá **borrarlas**. No apliques ciegamente contra una DB con datos que no gestiona la app.

Regla de oro: lee siempre el `upgrade()/downgrade()` generado antes de aplicarlo y ajusta lo que haga falta.

Crear una migración vacía (sin autogenerate)

Cuando quieres escribir los cambios a mano (renombrados, migración de datos, DDL que autogenerate no cubre):

```
uv run alembic revision -m "descripcion del cambio"
```

Esto crea el archivo sin conectarse a la DB; rellenas tú `upgrade()` y `downgrade()` con operaciones `op.*` (`op.create_table`, `op.add_column`, `op.alter_column`, `op.drop_column`, `op.execute("SQL...")`, etc.).

PROF

¿Contra qué base de datos corre?

Por defecto, `env.py` usa la misma URL que la app (desde `settings`, es decir tu `.env`). Para apuntar a otra DB sin tocar el `.env`, exporta `DATABASE_URL`:

```
# Contra el MySQL de test que corre en Docker (puerto 3307)
DATABASE_URL="mysql+asyncmy://root:pearsonhardman@127.0.0.1:3307/buholegal" \
  uv run alembic upgrade head

# Contra un SQLite local de prueba (rápido, sin infra)
DATABASE_URL="sqlite+aiosqlite:///./prueba.db" \
  uv run alembic upgrade head
```

Situaciones frecuentes

- **"Target database is not up to date"** al autogenerar → te falta aplicar migraciones: `uv run alembic upgrade head` y reintenta.
- **La migración autogenerada sale vacía** → no hay diferencias entre modelos y DB (todo está sincronizado). Puedes borrar ese archivo.
- **Adoptar Alembic en una DB que ya tiene las tablas** (creadas antes por `create_all`, por ejemplo): marca la revisión como aplicada sin ejecutar el DDL con `stamp`:

```
uv run alembic stamp head
```

- **Múltiples "heads"** (dos ramas de migraciones, típico tras un merge de git) → se resuelve fusionándolas:

```
uv run alembic merge -m "merge heads" <head1> <head2>
```

Relación con los tests

Los tests **no** usan Alembic: crean el esquema con `Base.metadata.create_all` sobre SQLite en memoria (ver `tests/conftest.py`). Es a propósito —así los tests son rápidos e independientes del historial de migraciones. Alembic es para las DB reales (dev, test en Docker, producción).